Normalization bounds and a Church STLC reduction for PTS-rk

October 20, 2026

Abstract

This development gives an executable normalization procedure for the original annotated PTS-rk calculus and proves explicit bounds for its operation cost. A semantic bound d on redex-head ranks guarantees normalization in d bottom-up greedy passes, with an elementary cost bound when d is fixed. For closed well-typed terms, a conservative rank bound can be computed in linear traversal cost and yields a tower bound whose height is linear in the input encoding size. The central proofs handle substitution-created redexes, lambdas exposed during a pass, and reduction inside annotations. A separate Church-style simply typed lambda calculus embeds into PTS-rk with preservation and reflection of beta-convertibility. This document explains the infrastructure, the proof dependencies, the correction to an earlier estimator argument, and the distinction between checked cost bounds and the external complexity-theoretic lower bound.

1 Scope and principal results

The library `PTSrk.NormalizationComplexity` imports the original PTS-rk kernel. Its terms are `Syntax.Term nat n`, its typing relation is `Typing.typ` instantiated with `Ranks.RankSorts`, and its reduction is the original beta relation. It introduces neither an alternative typing judgment for the target nor an annotation-erasing syntax. It is independent of both arithmetic interpretations and of the set model.

Write U_r for sort r. The characteristic rules are $U_r : U_0$ and

$$\frac{\Gamma \vdash A : U_a \qquad \Gamma, x : A \vdash B : U_b}{\Gamma \vdash \Pi(x : A).B : U_{\max(a+1,b)}}.$$

The increase on the domain side is the source of the strict decrease used in normalization. However, the rank of a function is determined by its whole product type, including the codomain; the annotation's rank alone does not determine it.

Let $N(t)$ be the number of syntax nodes and $s(t)$ the encoding size defined below. Set

$$\mathrm{tower}_0(x) = x, \qquad \mathrm{tower}_{h+1}(x) = 2^{\mathrm{tower}_h(x)}.$$

The public theorems have three complementary roles.

Theorem 1.1 (Fixed semantic rank)**.** *If Γ is well formed, $\Gamma \vdash t : T$, and every redex occurrence of t has head rank at most d, then `normalizeCosted d t` returns a term u such that $t \to_\beta^* u$, u is beta-normal, and $\Gamma \vdash u : T$. Its charged cost is at most*

$$\mathrm{tower}_{2d+1}\big(N(t) + 5d + 2\big).$$

This is `fixed_rank_normalization`. The bound is elementary for each fixed d; the theorem also applies to open terms in well-formed contexts.

Theorem 1.2 (Automatic normalization)**.** *For every closed original-PTS typing derivation* $\vdash t : T$*, the executable procedure* `automaticNormalizer t` *returns a beta-reachable normal form of the same original type* T*. With* $s = s(t)$*, its charged total cost is at most*

$$\mathrm{tower}_{4s+3}(18s + 8).$$

The procedure computes its own sufficient bound, at cost at most $7s$*; the bound has magnitude at most* $2s + 1$*. These statements are* `normalization_in_TOWER` *and* `rank_bound_in_linear_time`*.*

The third main result, `ChurchSTLC.church_stlc_reduction`, gives a conversion-preserving and conversion-reflecting embedding of closed, independently simply typed Church terms. Its exact size equation and its role in the lower-bound argument are discussed below. None of these theorems assumes an estimator-soundness, descent, or termination contract.

2 Infrastructure and source organization

The development separates properties of raw operations from the semantic argument establishing a sufficient number of passes. This separation is useful: a total bounded attempt can have a proved cost bound even when its budget is too small for success.

The source groups are as follows.

- `Rank.v` translates the reference and optimized estimators from `rank.agda`, including their auxiliary contexts and application spines. `Bounds.v` introduces the input measures and tower functions.
- `Greedy.v` defines the reducer, its iterates, and the normal-form test. `Runtime.v` supplies executable costed versions and proves their value projections equal the uncosted operations.
- `RedexRanks.v` defines semantic occurrence ranks using original typing derivations. `Morphisms.v` and `Safety.v` provide typed substitutions, equivalent structural rank bounds, and transport through context changes.
- `Descent.v` proves strict decrease; `BoundedNormalization.v` packages fixed-rank normalization with the executable cost bound.
- `Literals.v`, `SortBounds.v`, and `AutomaticBound.v` derive a sufficient bound from the input syntax. `Normalization.v` composes the resulting automatic normalizer and its total cost.
- `EstimatorChecks.v` proves the counterexample to the earlier automatic bound. `STLCEmbedding.v` contains the independent source calculus and the two-way conversion theorem. `Audit.v` exposes the public types and transitive assumptions.

All binders retain their annotations. The original de Bruijn infrastructure makes scopes explicit in the term type: renaming changes variable indices, simultaneous replacement maps variables to terms, and lifting extends these maps under a binder. Consequently a proof about substitution must account for both the binder's annotation and the shifted replacement terms in its body.

3 Semantic rank and the greedy strategy

3.1 The rank being decreased

`TypeRank G t r` means that there is a type T with

$$\Gamma \vdash t : T \qquad \text{and} \qquad \Gamma \vdash T : U_r.$$

At an occurrence $(\lambda(x : A).b)\,u$, `RedexRank` records original derivations assigning the head a product type $\Pi(x : U).V$ and that product the sort U_r. Its other constructors descend into either side of a product, lambda, or application. In particular, redexes in annotations count, and a body occurrence is interpreted in the extended context. `RedexBound G t d` universally bounds these occurrence ranks by d. It says nothing about a fuel value already succeeding.

The original uniqueness-of-typing and conversion results show that these ranks are unique on well-typed terms in well-formed contexts. The rank is preserved by typed substitution and reduction, so it does not depend on which convertible presentation of the product type is used.

Product inversion gives the decisive inequalities. If a function has rank $r = \max(a + 1, b)$, then its argument has rank $a < r$, while the application's result has rank $b \leq r$. Strictness belongs to the argument; there need not be a strict decrease along the result of an application spine. Every redex rank is positive. Conversely, every non-normal well-typed term has a redex rank, so a zero bound implies normality (`zero_bound_normal`).

3.2 One pass

The function `greedy` first recursively reduces both children of every constructor. At an application it then inspects the *reduced* function:

$$g(f\,u) = \begin{cases} b'[u'/x], & g(f) = \lambda(x : A').b', \quad u' = g(u), \\ g(f)\,g(u), & \text{otherwise.} \end{cases}$$

Products and lambdas become products and lambdas of their recursively reduced annotations and bodies. This is a bottom-up parallel pass, not just the simultaneous contraction of redexes already visible in the input: reducing f can expose a lambda which is contracted immediately. `greedy_sound` proves $t \to_\beta^* g(t)$ by structural induction and the original congruence rules. The function `rounds d t` iterates this pass exactly d times.

The boolean `normalb` checks every subterm and rejects an application with a lambda at its head. Its soundness refers to the original one-step relation: when it returns true, no original beta step is possible. `normalizeWithin` runs a prescribed number of passes and returns `Some` only if this check succeeds. A `None` result expresses an insufficient certificate of normality, not divergence of the function.

4 Why a pass strictly decreases the bound

The proof of `greedy_strict_descent` establishes

$$\text{RedexBound}(\Gamma, t, d + 1) \quad \Longrightarrow \quad \text{RedexBound}(\Gamma, g(t), d)$$

for original well-typed terms in well-formed contexts. Three issues must be resolved before a structural induction can prove this statement.

4.1 Redexes created by substitution

A bound on the redexes already inside b and u is not by itself enough to bound the redexes of $b[u/x]$: a variable in function position can be replaced by a lambda. `Safety.v` therefore introduces `RankSafe`, a structural presentation of the same occurrence bound. On well-typed inputs, `safe_iff_bound` proves its equivalence with `RedexBound`; this is an induction interface, not a stronger input condition.

For simultaneous substitution, `SafeImages` controls both the internal redexes of each image and the type rank of an image that is itself a lambda. Typed substitution maps are lifted under binders, and type-rank uniqueness supplies reflection of rank across substitution. These facts prove the residual estimate `redex_bound_substitution`:

$$\begin{aligned} &\mathrm{RedexBound}((\Gamma, x : U), b, d), \qquad \mathrm{RedexBound}(\Gamma, u, d), \\ &\qquad\qquad \mathrm{TypeRank}(\Gamma, u, a) \\ \Longrightarrow \quad &\mathrm{RedexBound}(\Gamma, b[u/x], \max(d, a)), \end{aligned}$$

under the corresponding original typing and well-formedness hypotheses. The extra a controls precisely the new head redexes introduced at variable occurrences.

4.2 Heads exposed during the current pass

The application clause can contract a head that was not syntactically a lambda before the pass. `exposed_lambda_rank` proves that such a lambda cannot arise without a sufficiently large original redex on the function spine. If a non-lambda term of type rank r becomes a lambda, some original redex has rank $q \geq r$.

The proof follows the application spine. When its original head is a lambda, the root redex supplies the witness. Otherwise the induction hypothesis supplies an earlier redex, and the non-increase of result rank carries its bound outward. Thus `greedy_contracted_head_bound` bounds every contracted head by the original application's bound, including heads exposed through substitution.

4.3 Annotations and the final induction

Reducing a binder annotation from A to $g(A)$ also changes the context in which its body is typed. `Morphisms.v` derives context transport from the original context-reduction theorem; `safe_context_last_reduction` transports the structural rank bound. This is required in both the lambda and the product cases, even if the operational term outside the annotation is unchanged.

Strict descent and bounded rounds. Induct on the original syntax using `RankSafe`. The hypotheses give the reduced children bound d. In a contracting application, the exposed-head result bounds the contracted function's rank by $d + 1$. Its argument has strictly smaller rank and hence rank at most d. The residual substitution estimate therefore gives $\max(d, a) = d$. In the non-contracting case there is no new head redex. For binders, transport the body into the reduced annotation's context. This proves `greedy_safe_descent`, then the equivalence of the two bound presentations gives strict descent.

Induction on d now proves `bounded_rounds_normal`. The base case uses positivity of redex ranks; the successor case combines one strict descent with original subject reduction. No separate normalization assumption enters this argument. □

5 Size growth and executable cost

5.1 Encoding conventions

`nodes` counts each term constructor once, including those in annotations. `encodingSize` also counts unary sort labels and unary de Bruijn indices: $s(U_r) = r + 1$, and a variable has size

one plus the length of its unary index. For a binary constructor, including a binder with its annotation and body, size is one plus the sizes of both children. Thus $N(t) \le s(t)$.

The traversal model charges linked-context lookup by the index length. Rank arithmetic and maxima have fixed word-operation charges; the bounds on computed ranks justify logarithmic-width words for closed inputs in this encoding. The reducer model charges constructor operations, callback work, and every lifting or shift performed during substitution. The natural-number cost components are ghost accounting: their own unary evaluation is not part of the claimed charge.

5.2 Raw size estimates

Renaming preserves the node count. Simultaneous replacement multiplies it by at most a bound on the sizes of the images, giving in particular

$$N(b[u/x]) \le N(b)N(u).$$

This estimate, combined with induction on the syntax, proves

$$N(g(t)) \le 2^{N(t)}, \qquad N(\mathrm{rounds}(k,t)) \le \mathrm{tower}_k(N(t)).$$

The application case uses the multiplicative substitution bound when the reduced head is a lambda. These size inequalities hold on raw syntax; they do not imply that k is sufficient for normality.

5.3 Charging the actual operations

`Runtime.v` returns a pair consisting of a value and a cost for renaming, substitution, greedy reduction, iteration, and the normal-form check. Projection lemmas identify the first component with the corresponding original operation. This rules out proving a bound for an unrelated abstract reducer.

Under a binder, a substitution image must be shifted. The code explicitly runs the costed renamer and adds its cost. With $L(n) = n(1+n)+1$, the resulting substitution estimate is

$$C_{\mathrm{subst}}(b,u) \le N(b)\big(1 + N(b)L(N(u))\big).$$

Together with the raw size bounds, this yields $C_g(t) \le 2^{8N(t)}$. To compose passes while also accounting for their growing inputs, the proof bounds *output size plus accumulated work* by iterations of $W(n) = 2^{16n}$. The elementary comparison $W^k(n) \le \mathrm{tower}_{2k}(n+4k)$ then absorbs loop overhead. The final normal-form test has cost at most twice its input's node count. `normalizeCosted_elementary` consequently gives

$$C_{\mathrm{attempt}}(k,t) \le \mathrm{tower}_{2k+1}(N(t) + 5k + 2).$$

This bound holds whether the bounded attempt returns `Some` or `None`. Combining it with the independently proved sufficiency of d passes gives the fixed-rank theorem.

6 Obtaining a sufficient bound from the syntax

6.1 The estimator and its limitation

`Rank.v` preserves the original Agda estimator and its optimization sequence. Its auxiliary context `RC` has declarations and let-declarations; these are bookkeeping for the estimator, not extra PTS constructors. The reference algorithm carries application lists. The simplified estimator discards certain malformed-spine checks; successive optimizations pair results, cache context values, and compute all fields in one traversal.

`rankPair_correct`, `rankCached_correct`, and `traversal_correct` show that these optimizations agree with the *simplified* estimator. They do not assert equality with the reference algorithm on arbitrary raw syntax. The optimized estimator has magnitude at most $2s$ and abstract two-pass traversal cost at most $10s$.

An earlier proposed bridge from maximum abstraction-domain rank B to semantic redex-head bound $B + 1$ is false. `EstimatorChecks.v` gives the following closed counterexample:

$$\begin{aligned} B &= \Pi(_ : U_0).U_0, \\ A &= \Pi(_ : B).U_0, \\ t &= \Pi(x : A).\Pi(y : B).\big((\lambda(z : U_0).x)\, U_0\big)\, y. \end{aligned}$$

The term has type U_3 and its optimized estimate is zero, but the displayed lambda has product type $\Pi(z : U_0).A$ of rank two. Hence t is not bounded by `estimateRank t + 1`. The theorem `estimator_does_not_bound_head_ranks` proves this exact failure. It does not show that the estimator-budget attempt fails on this input, or that the overall tower bound is false. A smaller-budget success theorem would need a different argument.

6.2 A conservative repair

Let $M(t)$ be the maximum sort literal occurring anywhere in t. `Literals.v` proves that renaming preserves this maximum and forward beta reduction cannot increase it. Substitution is bounded by the maxima of the body and the argument. These are forward-reduction facts; arbitrary beta expansion can introduce larger literals.

The main semantic bridge, `sort_rank_size`, is deliberately stronger than a statement about the input term alone. It bounds the sort of a term after any suitably typed simultaneous substitution and any application spine whose arguments satisfy the same literal bound. If all the required literals are at most m, the resulting sort rank is at most $m + N(t)$.

This uniformity makes the lambda case work: after consuming the first argument, the induction continues on the body with an extended substitution and the remaining spine. The variable case uses the substituted context declaration. Confluence supplies a common reduct when a type is convertible to a product or a sort; literal monotonicity is then applied to forward reductions, avoiding an invalid bound through arbitrary conversion.

`AutomaticBound.v` combines this sort bound with bounds on context declaration sizes and type-rank inversion. For a closed term it obtains

$$d(t) = 1 + M(t) + N(t), \qquad \mathrm{RedexBound}(\varnothing, t, d(t)), \qquad d(t) \le 2s(t) + 1.$$

This is `conservativeRank`. The bound counts all syntactic binders, including products. Its costed implementation uses a literal scan and a node-count traversal and has charge at most

$7s$. Substituting $d \le 2s+1$ and $N \le s$ into the bounded-attempt cost gives $\mathrm{tower}_{4s+3}(11s+7)$. Charging the rank calculation and the final composition yields the public $\mathrm{tower}_{4s+3}(18s+8)$ bound.

The normalizer is a total function on raw closed syntax. Its result type still permits `None`; original closed well-typedness proves that branch unreachable. No typing decision procedure is part of the timed operation, and the theorem does not require a supplied successful fuel certificate.

7 The independent Church STLC embedding

`STLCEmbedding.v` defines simple types $A ::= o \mid A \to A$, their order, an environment, a Church-style typing relation, and ordinary beta rules. It shares scoped raw syntax with PTS-rk but its typing constructors make no appeal to target typing. The distinguished variable o denotes the ground type and has no source term binding. Lambda annotations are codes of simple types:

$$\begin{aligned} \ulcorner o \urcorner &= o, \\ \ulcorner A \to B \urcorner &= \Pi(_ : \ulcorner A \urcorner).\ulcorner B \urcorner, \\ \mathrm{ord}(o) &= 0, \\ \mathrm{ord}(A \to B) &= \max(\mathrm{ord}(A)+1, \mathrm{ord}(B)). \end{aligned}$$

Indices in the codomain are shifted under the product binder. `code_typed` places the code of A in $U_{\mathrm{ord}(A)}$ when $o : U_0$. Induction on independent source typing then proves `typing_preserved`. Substitution into a nondependent codomain returns the original type code, which supplies the application case.

7.1 Why reflection is stronger than simulation

Source beta steps are target beta steps, so preservation of conversion is immediate. Reflection requires ruling out extra target computation. Every type code is beta-normal. A `Shape` predicate describes source term forms with these annotations and is preserved by typed-shape substitution and renaming. Thus any target step starting from a source shape is a source step and leaves another source shape: `step_reflected`.

For a general conversion, an arbitrary zigzag might leave the source fragment on its expansion segments. The proof instead uses original confluence to obtain a common forward reduct. Every step on both forward paths reflects, so the paths give source convertibility. This is `conversion_reflected`.

To turn the distinguished ground-type variable into a closed PTS term, define $\mathrm{close}(t) = \lambda(o : U_0).t$. The source input is closed in term variables, although its raw syntax retains the one ground-type slot. Reduction of the wrapper can only reduce its body: its annotation U_0 is normal. Common-reduct reasoning therefore also reflects conversion between the closed wrappers.

Theorem 7.1 (Closed Church reduction)**.** *For source terms t, u independently derivable at the same simple type A with no free term variables, the terms* $\mathrm{close}(t)$ *and* $\mathrm{close}(u)$ *are closed original-PTS inhabitants of* $\Pi(o : U_0).\ulcorner A \urcorner$*, and*

$$t =_\beta^{\mathrm{STLC}} u \quad \Longleftrightarrow \quad \mathrm{close}(t) =_\beta^{\mathrm{PTS}} \mathrm{close}(u).$$

Moreover, their combined encoding size increases by exactly four:

$$s(\mathrm{close}(t)) + s(\mathrm{close}(u)) = s(t) + s(u) + 4.$$

This is the conjunction packaged by `ChurchSTLC.church_stlc_reduction`*.*

The size calculation counts every existing Church annotation and adds one lambda and one sort per term. It avoids any claim about inferring or reconstructing explicit annotations from Curry-style inputs. It is an exact syntax-size statement, not a machine-time certificate for a serializer.

8 Discussion and verification boundary

The development makes the normalization proof quantitative without modifying the calculus. Its most consequential ingredients are the semantic distinction between a function's rank and its domain's rank, the residual substitution bound, and the tracing of exposed lambdas to original spine redexes. Keeping annotations visible forces context transport into the proof and ensures that the returned term is normal for the original reduction relation.

The numerical constants are safe envelopes rather than claims of optimality. Fixed semantic rank bounds the number of greedy passes; it does not make each pass constant cost. Conversely, a raw cost theorem for a bounded attempt proves neither normality nor success. The final theorems combine these two logically separate obligations.

For the lower bound, Nguyen's Simply typed convertibility is TOWER-complete even for safe lambda-terms[1] (LMCS 20(3:21), 2024, Theorem 1) supplies the external STLC hardness theorem under elementary many-one reductions. Its input discussion includes Church-style terms with annotations counted in size. Combining that mathematical theorem with the checked embedding supports the manuscript's lower-bound argument for closed well-typed beta-convertibility. Rocq checks the embedding, not the cited hardness proof; the hardness result has not been postulated as an axiom.

The repository does not formalize Turing-machine complexity classes, a machine-level encoding and implementation theorem, or the external STLC hardness proof. Accordingly, `normalization_in_TOWER` is the explicit operation-cost upper bound stated here, not an unconditional machine-level TOWER-completeness theorem. A binary-label representation would also require a separate bit-cost analysis. Executing Gallina's unary cost counters is not covered by the operation charge.

The sharper original-estimator budget remains an optional refinement; the conservative repair already proves the public upper bound. A complete machine-level complexity formalization would additionally connect the syntax encoding and executable operations to a machine cost model and formalize the external lower bound.

[1] `https://arxiv.org/html/2305.12601v4`

8.1 Reproducing the proof audit

The Dune theory depends only on `PTSrk`, `Stdlib`, and `Equations`. Use the pinned repository environment: OCaml 4.14.2, Dune 3.23.1, Rocq 9.2.0, Stdlib 9.1.0, and Equations 1.3.2+9.2. From the repository root, in that environment, run:

```
dune build @theories/NormalizationComplexity/audit
```

`Audit.v` contains 95 transitive `Print Assumptions` commands together with checks of the principal theorem types. The retained fresh audit and recursive kernel checks for the completed STLC extension reported closed assumptions for all 95 audited declarations. Loaded-library assumptions are distinguished from assumptions of a particular theorem; the audit does not infer a theorem dependency merely from a library being loaded.

A new assumption report requires a fresh absolute Dune build directory: cached targets do not rerun `Print Assumptions`. The repository's `tools/check-local-ci.sh` additionally runs the complete staged driver, including a pristine upstream build. Current commands and dated evidence are indexed in `docs/DirectR-CI.md` and the live handoff state; historical logs preserve their original paths. PDF compilation checks this exposition's typesetting and is separate from the Rocq proof audit.